Relative path: detection/gru_autoencoder.py
Filename: gru_autoencoder.py Extension: .py

```
"""
OASIS v2 — Layer 1 core: GRU Sequence Autoencoder (pure NumPy)
================================================================

Why GRU, not Transformer / graph (ARCHITECTURE.md Section 5)
------------------------------------------------------------
Sessions are short (command_sequence is typically well under 20 actions).
A Transformer's main advantage — long-range attention over long sequences —
doesn't pay for itself at this length, and it costs more to train and to
explain to a SOC analyst than "the model couldn't reconstruct this action
sequence." A graph-based model would be the right call if the detection
target were entity-resource *relationship* structure across many entities at
once; here the unit of analysis is a single entity's single session, so a
sequence model is the better-fit tool, not a bigger one.

Why an autoencoder, not a supervised classifier here
----------------------------------------------------
This is Layer 1's answer to class imbalance (ARCHITECTURE.md Section 3):
train ONLY on normal sessions, score anomalies by reconstruction error. The
model never needs to see a labeled attack at train time.

Implementation note (read this before assuming a bug)
-----------------------------------------------------
This sandbox has no network access and no `torch` installed, so this is a
from-scratch NumPy implementation of a GRU encoder-decoder with manual
backprop-through-time (BPTT), rather than `torch.nn.GRU`. The math is the
standard GRU cell equations; single-session (batch size 1) training loop,
which is fine at this dataset size (~700 sessions, sequences <20 long) but is
the first thing to swap for `torch.nn.GRU` + minibatching if this needs to
scale past a hackathon dataset. That swap is a drop-in replacement for
`GRUAutoencoder` below — the rest of the pipeline (detection/model.py
onward) only depends on `.reconstruction_error(session)`, not on how it's
computed internally.
"""
from __future__ import annotations

import numpy as np


def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -30, 30)))


class GRUAutoencoder:
    def __init__(self, vocab_size: int, embed_dim: int = 8, hidden_dim: int = 16,
                 max_len: int = 15, seed: int = 42):
        self.vocab_size = vocab_size
        self.E = embed_dim
        self.H = hidden_dim
        self.max_len = max_len
        rng = np.random.default_rng(seed)

        def w(shape):
            return rng.normal(0, 0.1, size=shape)

        self.emb = w((vocab_size, embed_dim))

        # encoder weights
        for g in ("z", "r", "h"):
            setattr(self, f"enc_W{g}", w((hidden_dim, embed_dim)))
            setattr(self, f"enc_U{g}", w((hidden_dim, hidden_dim)))
            setattr(self, f"enc_b{g}", np.zeros(hidden_dim))
        # decoder weights (separate params)
        for g in ("z", "r", "h"):
            setattr(self, f"dec_W{g}", w((hidden_dim, embed_dim)))
            setattr(self, f"dec_U{g}", w((hidden_dim, hidden_dim)))
            setattr(self, f"dec_b{g}", np.zeros(hidden_dim))
        self.Wo = w((embed_dim, hidden_dim))
        self.bo = np.zeros(embed_dim)
        self.start_token = np.zeros(embed_dim)

        self._param_names = (
            [f"enc_W{g}" for g in "zrh"] + [f"enc_U{g}" for g in "zrh"] +
```

```
            [f"enc_b{g}" for g in "zrh"] + [f"dec_W{g}" for g in "zrh"] +
            [f"dec_U{g}" for g in "zrh"] + [f"dec_b{g}" for g in "zrh"] +
            ["Wo", "bo", "emb"]
        )
        self._m = {p: np.zeros_like(getattr(self, p)) for p in self._param_names}
        self._v = {p: np.zeros_like(getattr(self, p)) for p in self._param_names}
        self._t = 0

    # -- GRU cell ------------------------------------------------------
    def _cell_forward(self, x, h_prev, prefix):
        Wz, Uz, bz = getattr(self, f"{prefix}_Wz"), getattr(self, f"{prefix}_Uz"), getattr(self, f"{prefix
}_bz")
        Wr, Ur, br = getattr(self, f"{prefix}_Wr"), getattr(self, f"{prefix}_Ur"), getattr(self, f"{prefix
}_br")
        Wh, Uh, bh = getattr(self, f"{prefix}_Wh"), getattr(self, f"{prefix}_Uh"), getattr(self, f"{prefix
}_bh")
        z = sigmoid(Wz @ x + Uz @ h_prev + bz)
        r = sigmoid(Wr @ x + Ur @ h_prev + br)
        hbar = np.tanh(Wh @ x + Uh @ (r * h_prev) + bh)
        h = (1 - z) * h_prev + z * hbar
        cache = (x, h_prev, z, r, hbar)
        return h, cache

    def _cell_backward(self, dh, cache, prefix, grads):
        x, h_prev, z, r, hbar = cache
        Uh = getattr(self, f"{prefix}_Uh")
        Ur = getattr(self, f"{prefix}_Ur")
        Uz = getattr(self, f"{prefix}_Uz")

        dz = dh * (hbar - h_prev)
        dhbar = dh * z
        dh_prev = dh * (1 - z)

        dhbar_raw = dhbar * (1 - hbar ** 2)
        grads[f"{prefix}_Wh"] += np.outer(dhbar_raw, x)
        grads[f"{prefix}_Uh"] += np.outer(dhbar_raw, r * h_prev)
        grads[f"{prefix}_bh"] += dhbar_raw
        d_r_hprev = Uh.T @ dhbar_raw
        dr = d_r_hprev * h_prev
        dh_prev += d_r_hprev * r
        dx = getattr(self, f"{prefix}_Wh").T @ dhbar_raw

        dz_raw = dz * z * (1 - z)
        grads[f"{prefix}_Wz"] += np.outer(dz_raw, x)
        grads[f"{prefix}_Uz"] += np.outer(dz_raw, h_prev)
        grads[f"{prefix}_bz"] += dz_raw
        dh_prev += Uz.T @ dz_raw
        dx += getattr(self, f"{prefix}_Wz").T @ dz_raw

        dr_raw = dr * r * (1 - r)
        grads[f"{prefix}_Wr"] += np.outer(dr_raw, x)
        grads[f"{prefix}_Ur"] += np.outer(dr_raw, h_prev)
        grads[f"{prefix}_br"] += dr_raw
        dh_prev += Ur.T @ dr_raw
        dx += getattr(self, f"{prefix}_Wr").T @ dr_raw

        return dx, dh_prev

    # -- full autoencoder forward/backward on one sequence -------------
    def _forward(self, ids):
        T = len(ids)
        x_seq = [self.emb[i] for i in ids]
        h = np.zeros(self.H)
        enc_caches = []
        for t in range(T):
            h, c = self._cell_forward(x_seq[t], h, "enc")
            enc_caches.append(c)
        latent = h

        h = latent
        dec_caches = []
        outputs = []
        prev = self.start_token
        for t in range(T):
            h, c = self._cell_forward(prev, h, "dec")
            dec_caches.append(c)
```

```
        o = self.Wo @ h + self.bo
        outputs.append(o)
        prev = x_seq[t]  # teacher forcing

    return x_seq, outputs, enc_caches, dec_caches

def reconstruction_error(self, ids, per_step: bool = False):
    ids = ids[: self.max_len]
    if len(ids) == 0:
        return (0.0, []) if per_step else 0.0
    x_seq, outputs, _, _ = self._forward(ids)
    errs = [float(np.mean((o - x) ** 2)) for o, x in zip(outputs, x_seq)]
    mean_err = float(np.mean(errs))
    return (mean_err, errs) if per_step else mean_err

def train_step(self, ids, lr: float = 0.01):
    ids = ids[: self.max_len]
    if len(ids) < 1:
        return 0.0
    x_seq, outputs, enc_caches, dec_caches = self._forward(ids)
    T = len(ids)

    grads = {p: np.zeros_like(getattr(self, p)) for p in self._param_names}
    dh_dec = np.zeros(self.H)

    # each cache stores h_prev (not h_t); reconstruct h_t per decoder step first
    hs = []
    for t in range(T):
        x, h_prev, z, r, hbar = dec_caches[t]
        hs.append((1 - z) * h_prev + z * hbar)

    loss = 0.0
    d_emb = np.zeros_like(self.emb)
    for t in reversed(range(T)):
        o = outputs[t]
        x_true = x_seq[t]
        diff = o - x_true
        loss += float(np.mean(diff ** 2))
        do = (2.0 / self.E) * diff
        grads["Wo"] += np.outer(do, hs[t])
        grads["bo"] += do
        dh = self.Wo.T @ do + dh_dec
        dx, dh_dec = self._cell_backward(dh, dec_caches[t], "dec", grads)
        # dx flows to the teacher-forcing input = x_seq[t-1] embedding (or start token)
        src_id = ids[t - 1] if t > 0 else None
        if src_id is not None:
            d_emb[src_id] += dx
        # else: gradient w.r.t. start_token, kept fixed (not trained)

    # dh_dec now holds gradient w.r.t. decoder's initial hidden state = latent = encoder final h
    dh_enc = dh_dec
    for t in reversed(range(T)):
        dx, dh_enc = self._cell_backward(dh_enc, enc_caches[t], "enc", grads)
        d_emb[ids[t]] += dx

    grads["emb"] = d_emb
    loss /= T

    # Adam update
    self._t += 1
    b1, b2, eps = 0.9, 0.999, 1e-8
    for p in self._param_names:
        g = np.clip(grads[p], -5, 5)
        self._m[p] = b1 * self._m[p] + (1 - b1) * g
        self._v[p] = b2 * self._v[p] + (1 - b2) * (g ** 2)
        mhat = self._m[p] / (1 - b1 ** self._t)
        vhat = self._v[p] / (1 - b2 ** self._t)
        setattr(self, p, getattr(self, p) - lr * mhat / (np.sqrt(vhat) + eps))

    return loss
```